



CHECKPOINT 2

// checkpoint 2 · cine fiap · cinco questões em escada, numa api de verdade

Mobile Development & IoT

Prof. Paulo Reis

O que você vai construir

FIAP

O Cine FIAP tem uma bilheteria digital. Ela é uma **API**: um programa que fica no ar esperando pedidos e devolvendo respostas em JSON.

A API inteira já está escrita: rotas, servidor, documentação e um **banco SQLite de verdade**. Você não mexe em nada disso.

O seu trabalho são **cinco questões em escada**: a primeira é uma função de três primitivos, e cada questão seguinte acrescenta exatamente uma coisa que já vimos em aula. Cada questão que você termina **acende uma rota** no Swagger.

Quem faz o quê

FIAP



A rota pega o que chegou, entrega para a sua função e traduz a resposta em código HTTP. É por isso que dá para testar tudo com Jest sem nunca abrir o navegador.

A rota não decide nada. Toda regra do cinema mora nas suas funções.

O mapa dos arquivos

Você só toca no . Cada arquivo de questão vem **só com comentários e helpers**: a função, da assinatura ao return, é você quem escreve, e o comentário aponta o teste que mostra como ela é chamada.

```
boilerplate/
├── src/
│   ├── api/                pronto, não mexa (leia rotas.ts)
│   ├── dados/repositorio.ts pronto, não mexa (o SQLite, leia o SQL)
│   ├── tipos.ts             Q3, Q4 e Q5 escrevem os tipos aqui
│   └── servicos/
│       ├── precoDoIngresso.ts  Q1
│       ├── poltronaValida.ts   Q2
│       ├── listarSessoes.ts    Q3
│       ├── venderIngresso.ts   Q4
│       └── fechamentoDoDia.ts  Q5
├── tests/                  leia à vontade, não mude
├── COMO-RESOLVER.pdf       o método: primeiro a estrutura, depois os dados
└── package.json             author, rm (e author2, rm2 na dupla)
```

A escada, antes de qualquer detalhe

Questão	Função	Acrescenta o quê	Vale
Q1	<code>precoDoIngresso</code>	uma função tipada, três primitivos, uma conta	1.0
Q2	<code>poltronaValida</code>	as nativas de texto do Checkpoint 1	1.5
Q3	<code>listarSessoes</code>	o primeiro tipo seu + <code>map</code>	2.5
Q4	<code>venderIngresso</code>	união discriminada + usar as suas Q1 e Q2	3.0
Q5	<code>fechamentoDoDia</code>	agregação, chave dinâmica, <code>null</code>	1.5

Typecheck limpo vale **0.5**. Total: 10.0.

Fazendo na ordem, o validador marca: Q1 = **1.0** · até Q2 = **2.5** · até Q3 = **5.0** · até Q4 = **8.3** · tudo = **10.0**.

os tipos das questões que você não fizer também valem sozinhos: quem modela os quatro e para na Q3 já está com 7.0

O trabalho é em **dupla: no máximo duas pessoas, uma entrega por dupla**. Fazer sozinho também vale.

A identificação fica no `package.json`, e é ela que liga o zip às notas:

```
"author": "Nome de quem entrega",  
"rm": "1234567",  
"author2": "Nome da segunda pessoa, se houver",  
"rm2": "7654321"
```

- fez **sozinho**: preencha `author` e `rm`, deixe os outros dois vazios;
- fez **em dupla**: preencha os quatro. A nota vale para os dois RMs;
- `author` e `rm` vazios **zeram o checkpoint**, como no primeiro.

```
npm install      # uma vez, logo que descompactar
npm run dev      # sobe a API com hot reload · abra http://localhost:3001/docs
npm test        # os testes visíveis
npm run typecheck # o compilador confere os tipos
npm run validar  # a sua nota nos visíveis + o GUIA DE TIPAGEM
npm run db:resetar # zera o banco: ele volta cheio na próxima subida
```

Salvou um arquivo, **o servidor reinicia sozinho**. O banco é um **arquivo SQLite** (`cinefiap.db`): reiniciar não apaga venda, só o `db:resetar`. Comece pelo `npm run validar`: no projeto recém baixado ele dá **0.00** e lista o que falta.

Antes da primeira questão, leia o COMO-RESOLVER.pdf: primeiro a estrutura, depois os dados.

O que já vem pronto · não mexa

FIAP

Arquivo	O que é
<code>src/api/servidor.ts</code>	sobe o Express e o Swagger
<code>src/api/rotas.ts</code>	as rotas, já tipadas
<code>src/api/openapi.ts</code>	a descrição que o Swagger mostra
<code>src/dados/repositorio.ts</code>	banco SQLite de verdade (arquivo <code>cinefiap.db</code>), com o SQL ali para você ler
<code>src/tipos.ts</code> (primeira metade)	os tipos do Checkpoint 1 + <code>TipoIngresso</code> + <code>PedidoVenda</code>
<code>tests/</code>	os testes visíveis, as fixtures e dois atalhos de leitura

O que é seu

Arquivo	Questão
<code>src/servicos/precoDoIngresso.ts</code>	Q1
<code>src/servicos/poltronaValida.ts</code>	Q2
<code>src/servicos/listarSessoes.ts</code>	Q3, com a interface <code>SessaoResumo</code> no <code>tipos.ts</code>
<code>src/servicos/venderIngresso.ts</code>	Q4, com <code>Venda</code> e <code>ResultadoVenda</code> no <code>tipos.ts</code>
<code>src/servicos/fechamentoDoDia.ts</code>	Q5, com <code>Fechamento</code> no <code>tipos.ts</code>
<code>package.json</code>	<code>author</code> , <code>rm</code> e, em dupla, <code>author2</code> e <code>rm2</code>

Funções auxiliares e arquivos extras são bem-vindos: costumam deixar o código mais fácil de ler.

Q1 · precoDoIngresso

Uma função, três primitivos, uma conta. 0 aquecimento.

Q1 · as regras

A assinatura é **com você**: `tests/precoDoIngresso.test.ts` mostra como a função é chamada, e os helpers `horaDoHorario` e `arredondar2` já vêm prontos no arquivo.

- **meia-entrada paga metade** do preço base;
- às **18:00 ou depois custa 10% a mais**, e as 18:00 em ponto já conta. O horário é texto `'HH:MM'`:
`substring` pega a hora, `Number` transforma;
- o acréscimo entra **depois** do desconto, e arredonde para **duas casas**.

```
entra (30, true, '14:00') → sai 15
entra (30, false, '21:00') → sai 33
entra (45, true, '21:00') → sai 24.75
```

rota que acende: `GET /preco?precoBase=45&meiaEntrada=true&horario=21:00`

Q2 · poltronaValida

As nativas de texto que o Checkpoint 1 cobrou, agora a serviço de uma regra.

Q2 · as regras

FIAP

A assinatura é **com você**: `tests/poltronaValida.test.ts` mostra a chamada, e o helper `fileirasDaSala` já vem pronto. **A poltrona chega suja** (`' b4 '`): limpe espaço e suba para maiúscula antes de olhar. O formato é letra de fileira + número de 1 a 10:

Capacidade	Fileiras	Existe	Não existe
20	A e B	<code>B10</code> , <code>' b4 '</code>	<code>C1</code>
30	A até C	<code>C10</code>	<code>D1</code>
60	A até F	<code>F10</code>	<code>G1</code> , <code>A11</code> , <code>A</code>

as nativas que resolvem: `trim` · `toUpperCase` · `charAt` · `substring` · `Number`

Q3 • listarSessoes

O primeiro tipo modelado por você.

Q3 · o tipo `SessaoResumo`

FIAP

Antes da função, escreva a interface no `tipos.ts`. Eu não dou a tipagem: dou o exemplo de valor, e você modela.

```
{  
  "id": "1", "titulo": "Duna", "horario": "14:00", "sala": "IMAX",  
  "formato": "LEGENDADO", "vendidos": 2, "capacidade": 60, "status": "ABERTA"  
}
```

`sala`, `formato` e `status` não são texto solto: já existem `Sala`, `Formato` e `StatusSessao` na primeira metade do arquivo. **Use os três.**

lição do Checkpoint 1: interface é estrutura · o exemplo aponta o tipo, o exemplo não entra no código

Monta a grade do dia: um `SessaoResumo` para cada sessão recebida.

- a ordem da saída é a mesma da entrada;
- `titulo` vem de **dentro do filme**, não é a sessão inteira;
- `vendidos` conta só as vendas **daquela** sessão;
- `status`: `ENCERRADA` se cancelada, `ESGOTADA` se não sobra poltrona, `ABERTA` no resto. Cancelada ganha `ENCERRADA` **mesmo estando cheia**.

```
entra  a grade + vendas [ {sessaoId:'3', poltrona:'A3'}, {sessaoId:'3', poltrona:'B1'} ]
sai    [ ..., { id:'3', titulo:'Coringa', sala:'VIP', formato:'LEGENDADO',
                  horario:'19:30', vendidos:2, capacidade:20, status:'ABERTA' }, ... ]
```

Q4 • venderIngresso

O coração do checkpoint. E as suas Q1 e Q2 viram peças dele.

Q4 · os dois tipos

Venda, um ingresso já vendido:

```
{ "sessaoId": "1", "poltrona": "A3", "preco": 22.5, "tipo": "ESTUDANTE" }
```

ResultadoVenda, as duas respostas possíveis:

```
{ "tipo": "VENDIDO", "venda": { "sessaoId": "1", "poltrona": "A3", "preco": 22.5, "tipo": "ESTUDANTE" } }
```

```
{ "tipo": "RECUSADO", "motivo": "poltrona ocupada" }
```

Modele as duas possibilidades **num tipo só**, que obrigue quem recebe a olhar **tipo** antes de ler o resto. A rota em `src/api/rotas.ts` faz exatamente isso: **ela é a especificação do seu tipo.**

Q4 · a ordem das recusas

Confira **nesta ordem**: os testes cobram a primeira recusa que se aplica.



Q4 · montando a venda com as suas peças

FIAP

Passou por todas as recusas, a venda nasce **das suas Q1 e Q2**, que o arquivo já importa:

- o **preço base sai da sala**: `PADRAO` 30 · `IMAX` 45 · `VIP` 60;
- **meia-entrada** é todo tipo que não for `'INTEIRA'` (`ESTUDANTE`, `IDOSO` e `CRIANCA` pagam meia);
- o preço final é a sua **Q1** com o horário da sessão;
- a poltrona é guardada **limpa**: sem espaço e em maiúscula.

A idade mínima vem da classificação: `LIVRE` 0 · `DEZ` 10 · `DOZE` 12 · `QUATORZE` 14 · `DEZOITO` 18. A idade exata **já passa**: quem tem 18 entra no `DEZOITO`.

Os textos de recusa são **exatamente os cinco do fluxo**, minúsculos e sem acento: os testes comparam letra por letra.

Q4 · entra e sai

Uma venda que dá certo (Matrix, IMAX, 21:00, então meia de estudante com acréscimo noturno):

```
entra { sessaoId:'4', poltrona:' f7 ', idade:20, tipo:'ESTUDANTE' }  
sai   { tipo:'VENDIDO', venda:{ sessaoId:'4', poltrona:'F7', preco:24.75, tipo:'ESTUDANTE' } }
```

Uma que é recusada (Coringa é **DEZOITO**):

```
entra { sessaoId:'3', poltrona:'B2', idade:16, tipo:'INTEIRA' }  
sai   { tipo:'RECUSADO', motivo:'idade abaixo da classificacao' }
```

Repare na poltrona: entrou **' f7 '**, saiu **'F7'**. E repare no preço: 45 da IMAX, metade de estudante, 10% da noite. É a sua Q1 trabalhando dentro da Q4.

Q5 • fechamentoDoDia

O desafio. Agregação de verdade, como relatório de gente grande.

Q5 · o tipo Fechamento

FIAP

```
{  
  "ingressos": 3,  
  "receita": 181.5,  
  "ocupacaoPorSala": { "IMAX": 1, "PADRAO": 0, "VIP": 10 },  
  "sessaoMaisCheia": "3"  
}
```

`ocupacaoPorSala` tem **uma chave por sala** que teve sessão no dia. `sessaoMaisCheia` vale `null` num dia sem venda, e o tipo precisa aceitar as duas coisas.

- `ocupacaoPorSala`: porcentagem **inteira**. A conta é o vendido na sala dividido pela **soma das capacidades das sessões daquela sala**;
- `sessaoMaisCheia`: o id da que mais vendeu. Empate fica com a primeira da lista;
- `receita` com duas casas;
- **sessão cancelada não entra em nada**: nem receita, nem contagem, nem capacidade, nem na disputa. Se a única sessão de uma sala foi cancelada, a sala não aparece;
- duas sessões na mesma sala entram **na mesma conta**.

Duas vendas na sessão 3 (Coringa, VIP de 20 lugares, 66 cada por ser noturna):

```
entra  a grade + vendas [ A1 e A2 da sessão 3 ]
sai    { ingressos: 2, receita: 132,
        ocupacaoPorSala: { IMAX: 0, PADRAO: 0, VIP: 10 },
        sessaoMaisCheia: '3' }
```

A VIP fica com 10 porque foram 2 ingressos numa sala de 20 lugares. IMAX e PADRAO aparecem com zero porque tiveram sessão no dia, só não venderam. A sessão 5 é cancelada, então a capacidade dela não entrou em conta nenhuma.

A nota

Nada é subjetivo. O `npm run validar` do seu zip é o mesmo programa que corrige, com os mesmos pesos.

As cinco questões · 9.50

Cada questão soma a fração dos **testes** que passam vezes o peso, e Q3, Q4 e Q5 somam também os **tipos** que você modela, verificados **campo a campo**:

Questão	Testes	Tipos	Total
Q1 <code>precoDoIngresso</code>	1.00		1.00
Q2 <code>poltronaValida</code>	1.50		1.50
Q3 <code>listarSessoes</code>	2.00	<code>SessaoResumo</code> 0.50	2.50
Q4 <code>venderIngresso</code>	2.00	<code>Venda</code> 0.50 + <code>ResultadoVenda</code> 0.50	3.00
Q5 <code>fechamentoDoDia</code>	1.00	<code>Fechamento</code> 0.50	1.50

Nos tipos você leva a **fração das checagens que passar**: um `SessaoResumo` com sete de oito certas vale 0.44, não zero. O relatório diz **qual campo falhou**.

O guia de tipagem

O `npm run validar` traz junto um **guia de tipagem**, que funciona offline e aponta, por questão:

- **Parâmetros sem tipagem**, ou sem a tipagem **correta**;
- **Retorno sem tipagem**, ou com o tipo errado;
- **Variáveis sem tipagem forte no corpo da função** (`const`, `let`, `var` sem anotação);
- `any` no lugar de tipo de verdade;
- **magic strings**: escrever `'IMAX'` na mão onde existe `Sala.IMAX`. O enum existe para isso.

o guia não vale nota por si: ele existe para você corrigir antes que o typecheck e a sonda cobrem

Escondidos, typecheck e o que não tira nota

Além dos visíveis, a correção roda um **conjunto escondido que não vem no zip**: mesmas regras, outros números, **nada fora deste enunciado**. Solução certa, mesma nota.

Typecheck - 0.50, na escada: com `npm run typecheck` limpo, meio ponto; chegou perto, ainda vale:

Erros do <code>tsc</code>	0	1	2	3	4	5 ou mais
Ponto	0.50	0.40	0.30	0.20	0.10	0.00

O que NÃO tira nota: a função nativa que você usou (`filter`, `for`, `map`, laço, tanto faz), `console.log` esquecido, arquivos extras, nomes de variáveis, comentários.

As três regras que zeram

FIAP

1 - Identificação vazia. `author` e `rm` vazios no `package.json` zeram o checkpoint, para a dupla inteira. O validador mostra a seção IDENTIFICACAO logo no começo: confira que está verde antes de entregar.

2 - Renomear ou apagar os arquivos de questão. Rotas e testes importam por caminho: se `src/servicos/precoDoIngresso.ts` não existir com esse nome, a questão vale zero. Trocar `.ts` por `.js` cai aqui: o projeto é TypeScript.

3 - Mudar os testes ou o validador. Eu restauro `tests/`, `validar.min.js`, `jest.config.js`, `tsconfig.json`, `src/api/` e `src/dados/` a partir do original antes de corrigir. Mexer neles não muda a nota, só a sua impressão dela. Testes seus, crie em arquivo novo.

Antes de fechar o zip, rode `npm run validar`: a nota dos visíveis mais o guia de tipagem limpo é o melhor retrato da sua correção, que só acrescenta os escondidos, com as mesmas regras.

Depois apague a `node_modules` e compacte:

```
# sozinho
zip -r rm_1234567.zip . --exclude "node_modules/*"

# em dupla: os dois RMs no nome
zip -r rm_1234567_7654321.zip . --exclude "node_modules/*"
```

Entrega até domingo, 27 de setembro, 23h59, uma por dupla, no Teams.

Conversar sobre o problema com outras duplas, pode. **Trocar arquivo, não**: cópia entre duplas é o mesmo zip com nomes diferentes, e isso eu vejo de longe.

O que você pode consultar

Tudo. Documentação, anotações, as aulas, a internet.

Sobre assistente de IA: use se quiser, mas **entenda o que colou**. Na aula seguinte à entrega eu chamo algumas duplas para explicar uma decisão do próprio código, e "o ChatGPT escreveu" não é resposta.

O objetivo não é o zip. É você conseguir subir uma escada dessas sozinho: função pura, tipo modelado, composição. Isso é o que vai te servir no Checkpoint 3, no Semestre 2 e na primeira entrevista técnica que você fizer.

Bom checkpoint

Comece pela Q1. Ela é pequena de propósito.